1. ATIP as an algorithm: design note

This note turns a published procedure for classifying arguments into a program. It labels arguments with their type, first against human labels in the note-lab annotation project, later in-game against the player's own arguments.

1.1. Background: the PTA and ATIP

The *Periodic Table of Arguments* (PTA, version 3.1) classifies arguments into 50 types (Wagemans; see Section 1.11). In the PTA an argument consists of three statements:

- a **conclusion**, the claim being argued for;
- a **premise**, the reason given for it;
- a **lever**, the usually unstated statement that says *how* the premise makes the conclusion acceptable ("rising prices are a CAUSE of strikes").

Each statement has a **subject**, what it is about, and a **predicate**, what it says of the subject. An argument's type is the unique combination of three parameters:

- **form**: which elements the conclusion and premise share. There are four forms, written α, β, γ, δ (Section 1.5.4).
- **substance**: what kind of statements or terms are involved (Section 1.5.5).
- **lever**: the relation the lever statement asserts, named by a keyword in capitals such as CAUSE or ANALOGOUS (Section 1.5.6).

The *Argument Type Identification Procedure* (ATIP, version 6) determines these in six steps (Wagemans 2026): (1) label the textual elements, (2) reformulate the argument as "conclusion because premise", (3) determine the form, (4) the substance, (5) the lever, and (6) name the type. This note follows the same six steps.

Each type has a name and a symbol: "argument from cause", `Cau`. We write a classified argument as a **code** of form, substance and symbol: `α FF Cau` is an alpha argument whose conclusion and premise are both statements of fact, of the type argument from cause. Section 1.12 lists all 50 types.

1.2. Scope

In scope:

- Typing one argument, a single premise → conclusion pair, with a code, plus a confidence and the runner-up codes, and the canonical terms and transformations it was typed from. These fields are the **output record**; it has the same shape as the gold labels (Section 1.7).
- Measuring each part of the procedure against human labels.
- The shape of the output that the game consumes.

This automates the Initial Analysis of CAPNA, the Comprehensive Assessment Procedure for Natural Argumentation (Hinton and Wagemans 2023, Section 1.11): find an argument and type it with ATIP. Its later stages (process, reasoning, language) evaluate the typed argument, with questions that depend on the type ("is a SIMILAR to b?").

Out of scope:

- Judging whether an argument is *good*. That is a later step that reads this one's output: CAPNA's process, reasoning and language stages.
- How the game uses the output: deciding which of its keys an argument unlocks (sketched in Section 1.8 only as an interface).
- Explication of missing premises in free text (see Section 1.9). In the lab, reconstructed premises come from the human labels.

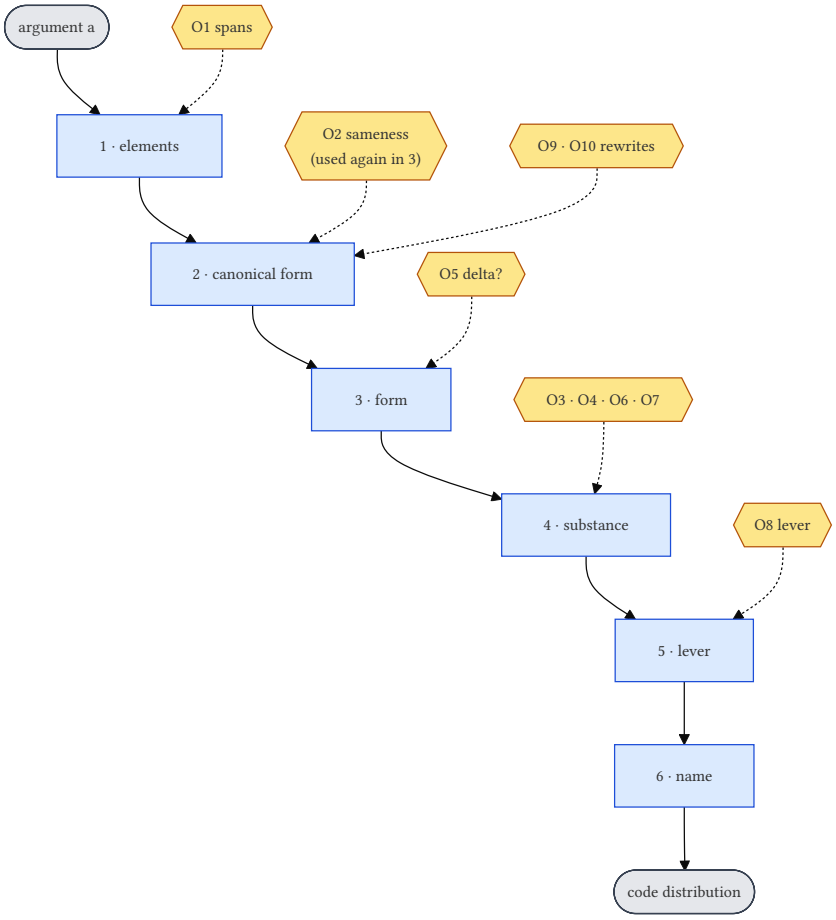
1.3. Principle: a deductive program with statistical oracles

ATIP is mostly a decision table. Steps 3 and 6 are lookups; steps 4 and 5 pick from small closed sets. So the program is deterministic, and every judgement it can't make by rule is an **oracle**: a named function from an argument a and its surrounding text ctx (both defined in Section 1.4) to a probability distribution over a small closed set:[1]

$$O_i : (a, \text{ctx}) \to \Delta(V_i), \quad |V_i| \leq 10$$

A rule is an oracle that returns a point mass.[2] Rules and models are therefore interchangeable per oracle, so each oracle has its own arms[3] in the lab (rule baseline, embedding model, NLI model, …). Every statistical judgement is localised, named and testable on its own.

There are ten oracles, O_1 to O_{10}. Each is introduced at the step that uses it, and Section 1.6 summarises them. O_9 is the exception to the closed-set rule: it generates candidate rewordings (Section 1.5.3). The whole pipeline, with deterministic steps as **blue rectangles** and oracles as **yellow hexagons**:



1.4. Input

1.4.1. The argument graph

A text is a graph with two kinds of node:

[1] V_i is the oracle's set of possible answers; $\Delta(V_i)$ is the set of probability distributions over it: one probability per answer, summing to 1.

[2] A distribution with all its probability on one answer: 1 for its answer, 0 for the rest.

[3] An arm is one way of answering an oracle, e.g. a rule baseline or a particular model with a particular configuration.

- **statements** S: anything that can be a conclusion, a premise or a lever;
- **connectors** K: one per PTA argument (one triad).

A connector $k \in K$ is defined by three role relations from statements to connectors:

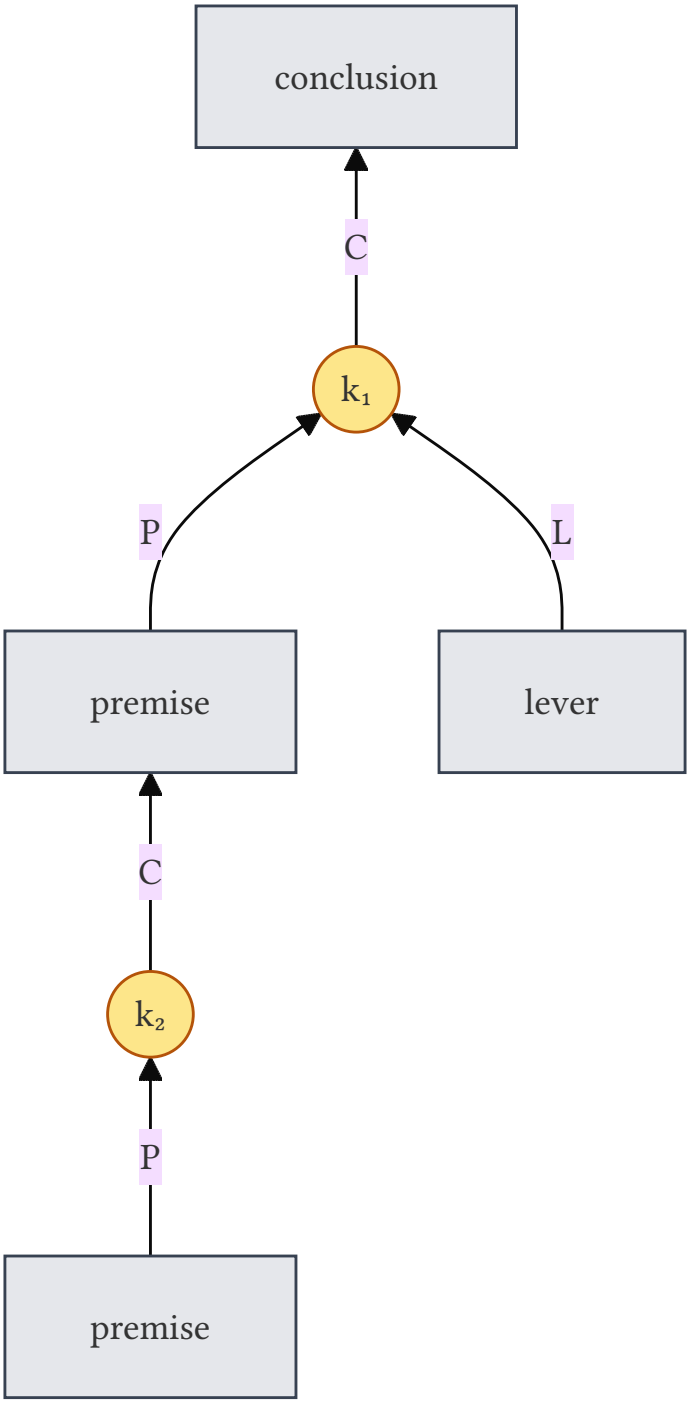
$$C, P, L \subseteq S \times K$$

$(s, k) \in C$ reads "s is the conclusion of k", and likewise P for premise and L for lever. Each connector has **exactly one** conclusion and **exactly one** premise, and **at most one** lever statement:

$$|C^{-1}(k)| = |P^{-1}(k)| = 1, \quad |L^{-1}(k)| \leq 1 \quad \text{for every } k \in K$$

A statement has no role of its own: its roles are the edges it takes part in, and it can take part in any number of them. The same statement can be the premise of one connector and the conclusion of another, which is what lets arguments nest.

Formally, a connector stands for an entailment relation: what goes into it entails its conclusion. When a conclusion C rests on two premises A and B jointly, the fair reading is $A \wedge B \vDash C$: one entailment, "C because A and B", by lever L. It is not two entailments, $A \vDash C$ and $B \vDash C$, since neither premise does the work alone. The cardinality above, exactly one premise, can't express this: two connectors would assert the two separate entailments, which is a different claim. How joint premises fit the graph is open.



This is the shape of the Argument Interchange Format (AIF; see Section 1.11): statements are its information nodes and connectors its inference nodes. The PTA adds the lever as a third kind of input.

The lever is usually implicit. A connector then has no lever statement, and its lever is only the keyword chosen in step 5. A lever gets a statement node when the text states it or argues about it; an attack on an inference is then an attack on its lever statement.

An argument, the unit the pipeline types, is one connector read off the graph: $a = (c, \rho, \mathrm{ctx})$ with c the conclusion of k, ρ (rho, for "reason") its premise and ctx the surrounding text. c and ρ are spans of text.

1.4.2. Sources

- **Lab:** the graph is labelled in INCEpTION, the annotation tool, on two span layers. `Statement` spans are the statements. A `Connector` span is a connector, anchored on the coherence marker ("because", "so") where there is one; it has three link features, `conclusion`, `premise` and `lever statement`, each pointing at `Statement` spans. A statement that is implicit in the text is a zero-width span with its wording in `note`. INCEpTION can't enforce the cardinalities above, so the converter checks them.
- **Game:** c and ρ come from an extra oracle over the player's line and the dialogue so far (Section 1.8).

1.5. Pipeline

Throughout, subscript c marks the conclusion and ρ the premise, and the conclusion always comes first.

Running example (made up, like all examples here):

"I'd say the minister did a great job, since she raised the university budget."

Probabilities in the examples are illustrative, not measured.

1.5.1. Step 1: elements

Split each statement into subject and predicate, and mark the surrounding elements. **O1** finds these spans.

$$E(c) = (s_c, p_c), \quad E(\rho) = (s_\rho, p_\rho), \quad M \subset \mathrm{ctx}$$

s is a subject and p a predicate. Write Su for the set of subject terms and Pr for the set of predicate terms, so $s_c, s_\rho \in \mathrm{Su}$ and $p_c, p_\rho \in \mathrm{Pr}$. M is the set of **surrounding elements**, which ATIP sorts into four kinds of marker:

- **coherence**: marks which statement is premise and which conclusion ("because", "since", "so");
- **structure**: relates the argument to others ("in addition", "secondly");
- **illocutive**: the arguer's stance towards the claim ("I'd say", "I don't believe");
- **perlocutive**: the effect the arguer intends ("you should accept that").

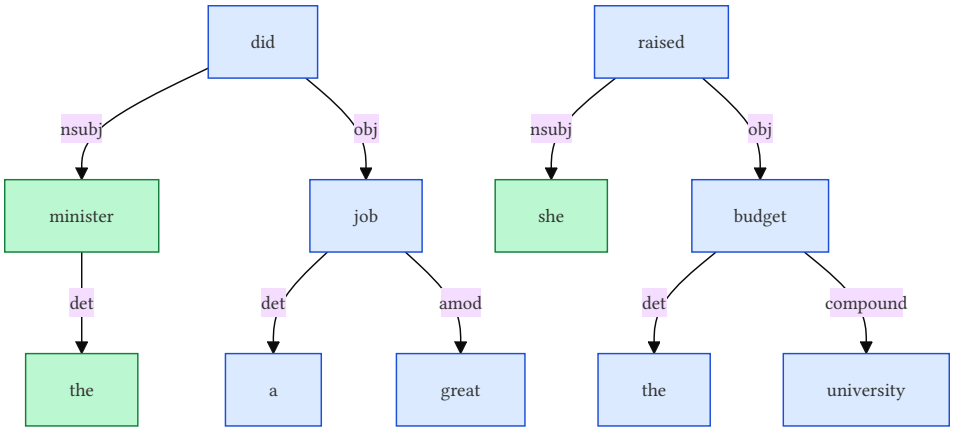
Gold has no subject/predicate spans: the labeller writes the canonical subjects and predicates, step 2′s output, as text (Section 1.7). So this is automated even in the lab, and scored through step 2′s fields. Rule baseline: dependency parse,[4] subject = the `nsubj`/`nsubj:pass` subtree, predicate = the rest of the clause; copulas[5] kept with the predicate. Markers come from a lexicon of the four kinds.

Step 1 also records q_ρ: the whole proposition the premise speaks of, if any ("Eurostat said *that the economy grew*"). Step 3 uses it to detect delta arguments. Rule baseline: the clausal complement of a reporting verb ("Eurostat said *that the economy grew*") or an anaphoric "so"/"that"/"this" ("Eurostat said *so*"), resolved to the clause it stands for. Otherwise q_ρ is empty.

Example. The human-labelled ("gold") spans give the conclusion "the minister did a great job" and the premise "she raised the university budget". Their parses, with **subject** and **predicate** words coloured:

[4] A parse that links each word to the word it grammatically depends on and labels the link: `nsubj` is a nominal subject, `nsubj:pass` the subject of a passive verb.

[5] Linking verbs such as "is", "are", "seems".




```
conclusion: {s_c: "the minister", p_c: "did a great job"}
premise:    {s_ρ: "she",          p_ρ: "raised the university budget"}
markers:
  - {text: "I'd say", kind: illocutive}
  - {text: "since",   kind: coherence}
```

1.5.2. Step 2: canonical form

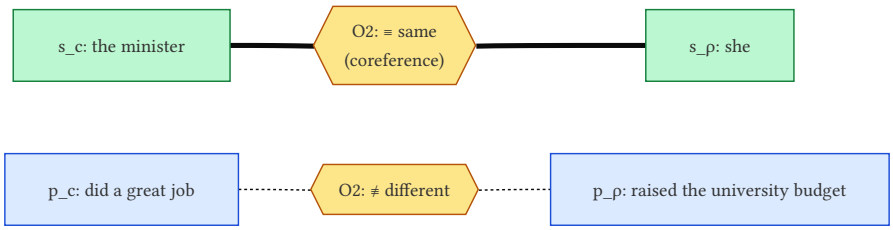
ATIP's step 2 rewrites the argument into the canonical sentence "conclusion because premise". Its transformations include moving the conclusion first, replacing the coherence marker by "because", dropping the other markers, resolving anaphora (replacing "she" by "the minister"), making implicit content explicit and switching between active and passive voice. Their purpose is to make shared subjects and predicates *literally* shared, because step 3 decides the form from what is shared.

Here the canonical form $\kappa(a)$ is not a rewritten string. It is the tuple from step 1 plus an equivalence relation $\equiv$ on terms:

$$\kappa(a) = \big((s_c, p_c), (s_\rho, p_\rho), \equiv\big)$$

$t_1 \equiv t_2$ means "t_1 and t_2 are the same subject or predicate after anaphora resolution, passivization and paraphrase"; $\not\equiv$ is its negation. Oracle **O2** judges it. Making $\equiv$ explicit means it doesn't matter whether two terms happen to share their wording. The canonical sentence in the output record is flattened from the tuple, using one representative wording per equivalence class.

Example. O2 compares the subjects and the predicates pairwise:



Flattening writes the same fields gold has (Section 1.7): the four canonical terms, and the transformations that changed a term's wording, each tagged with the statement it applied to. Each appears once, in any order. Transposition, the "because" substitution and omitted markers aren't listed: the conclusion and premise roles and the markers already record them. An $\equiv$ judgement between two different wordings is tagged `same term`, so O2's decisions show next to the rewrites.

The canonical sentence "conclusion because premise" and the lever written out are rendered from these fields, not stored.

```
canonical:
  conclusion: {subject: "the minister", predicate: "did a great job"}
  premise:    {subject: "the minister", predicate: "raised the university budget"}
transformations: ["anaphora (premise)"]
```

Delta reading. Some arguments rest on who said something or what would follow from it, not on its content: "the economy has grown, because Eurostat said so". For these the PTA splits both statements differently: the subject is the whole proposition. When q_ρ is non-empty, κ also produces this second split:

$$\kappa^{\delta}(a) = \big((c, \zeta), (q_\rho, Z)\big), \quad \zeta \in \mathbb{A} = \{A, U\}$$

In the conclusion, c as a whole is the subject and its predicate ζ is A (*acceptable*) or U (*unacceptable*); $\mathbb{A}$ is the set of the two. In the premise, q_ρ is the subject and Z is what the premise says of it ("was said by Eurostat"). ζ is usually implicit, so flattening writes it in brackets, as ATIP does: "the economy has grown [is acceptable (A)] because that was said by Eurostat". Step 3 decides which split is used.

Passivization works on the parse. In "the budget was raised by the minister", the subject is `nsubj:pass` = "the budget"; when there is an agent (`obl:agent` = "the minister"), the rule baseline also tries the active reading (subject "the minister"), so a shared subject isn't missed.

1.5.3. Rewriting: making shared terms visible

$\equiv$ can only judge terms that are already there. Some arguments share a term only after a rewording that creates new text. Kant writes "I need not think, if only I can pay: others will readily undertake the irksome work for me". As written the subjects differ ("I", "others"), which would make it γ. Reworded as "I will have others undertake the work for me", the premise shares the subject "I", and the argument is α. No judgement of sameness between the existing terms finds that: the shared term has to be produced.

ATIP's transformations therefore come in two kinds:

kind	transformations	handled by
selective	moving the conclusion first; replacing or dropping markers; resolving anaphora; recognising that two existing terms mean the same	rules, O1, O2
generative	making implicit content explicit; turning a rhetorical question into an assertion; switching voice beyond what the parse gives; nominalising ("being immature"); paraphrasing a term so it matches another	**O9**, checked by **O10**

Selective transformations only rearrange or compare what is in the text. Generative ones write new wording, so they need a proposer and a check.

O9: rewrite proposer. Given the argument and its context, a generative model proposes up to n candidate canonical pairs (c', ρ'). Each candidate lists the transformations it used, from ATIP's list plus nominalisation, and the source span each new phrase rewrites. O9 is not an oracle in the sense of Section 1.6: its output is text, not a choice from a closed set. Its candidates *become* the closed set the rest of the pipeline chooses from.

O10: rewrite check. A candidate is valid if it says the same as the original in context. For a rewording, an NLI model[6] checks entailment both ways; for explication, which adds content, only that the original and the context entail the rewrite:

$$P(\text{valid}(c')) = \min(\text{entail}(c, c'), \text{entail}(c', c))$$

Preference. ATIP switches voice "for highlighting a common subject or predicate", which implies preferring the reading that shares a term. So among valid candidates, γ is reported only when no candidate with $P(\text{valid}) \geq \theta$ shares a subject or predicate. Otherwise each candidate is one more branch of the composition (Section 1.5.8), weighted by $P(\text{valid})$.

Example. The Kant sentence, with illustrative numbers:

candidate	premise as reworded	transformation	$P(\text{valid})$	form
as written	others will undertake the irksome work for me	—	1.00	γ
c'_1	I will have others undertake the irksome work for me	activization	0.83	α
c'_2	my thinking will be done by others	passivization, nominalisation	0.61	γ

c'_1 is valid and shares "I", so γ isn't reported; the code comes from the α branch (here α PF Pra). Compare "it is so easy to be immature … I need not exert myself", where the predicates "is easy" and "need not exert myself" may already be the same: that is an ≡ judgement (O2), not a rewrite.

Visibility. Every candidate goes into the output record with its text, transformations, source spans and $P(\text{valid})$, and the chosen one is marked:

```yaml
rewrites:
  - premise: I will have others undertake the irksome work for me
    transformations: ["activization (premise)"]
    rewrites_span: "others will readily undertake the irksome work for me"
    p_valid: 0.83
    chosen: true
```

A reviewer can reject a rewrite and rerun from step 3. Gold records the labeller's own rewording in the same fields (Section 1.7), so the lab can measure whether O9′s candidates include it.

Limits. A generated rewrite can tidy an argument into one the author didn't make. That is why O9 is restricted to ATIP's transformations, each tied to a source span, and why its candidates are kept and scored rather than taken as given. A generative model is also the most expensive part of the pipeline; in-game use may need a smaller model or no generative step.

Arms and cost. Estimates for a laptop CPU, to be measured in the lab:

arm	what it is	per candidate
O9 rules	voice switches from the parse, nominalisation from a morphology table	1 ms
O9 seq2seq	Flan-T5-small or -base (80–250M parameters), fine-tuned to rewrite a premise to share a term with its conclusion; ONNX, quantised	50–150 ms

[6]Natural language inference: given a text and a hypothesis, the model scores whether the text entails the hypothesis.

arm	what it is	per candidate
O9 LLM	a 0.5–1.5B instruction-tuned model, 4-bit, via llama.cpp; the only arm that handles real paraphrase and explication	0.3–2 s
O10	DeBERTa-v3 NLI, xsmall to base, both directions	10–120 ms

O9 is the bottleneck, so it runs only when needed:

1. Run steps 1–3 with O2 alone. If a subject or predicate is already shared, or the argument is δ, skip O9.
2. Otherwise propose candidates with the rule arm and check them with O10.
3. Only if none passes, call a generative arm.

Cost then grows with the number of γ-leaning arguments, not with document length. Canonical forms are cached per statement, so a growing document or dialogue only pays for its new arguments.

1.5.4. Step 3: form

ATIP distinguishes four forms by what the conclusion and premise share. It writes them with schematic letters: a and b are subjects, X and Y predicates (X the conclusion's, Y the premise's), q a whole proposition and Z the premise's predicate in a delta argument. In our terms $a = s_c$, $b = s_\rho$, $X = p_c$, $Y = p_\rho$.

form	ATIP schema	shared	test here
α alpha	a is X because a is Y	subject	$s_c \equiv s_\rho, p_c \not\equiv p_\rho$
β beta	a is X because b is X	predicate	$p_c \equiv p_\rho, s_c \not\equiv s_\rho$
γ gamma	a is X because b is Y	nothing	$s_c \not\equiv s_\rho, p_c \not\equiv p_\rho$
δ delta	q is A/U because q is Z	the whole proposition	$c \equiv q_\rho$ (below)

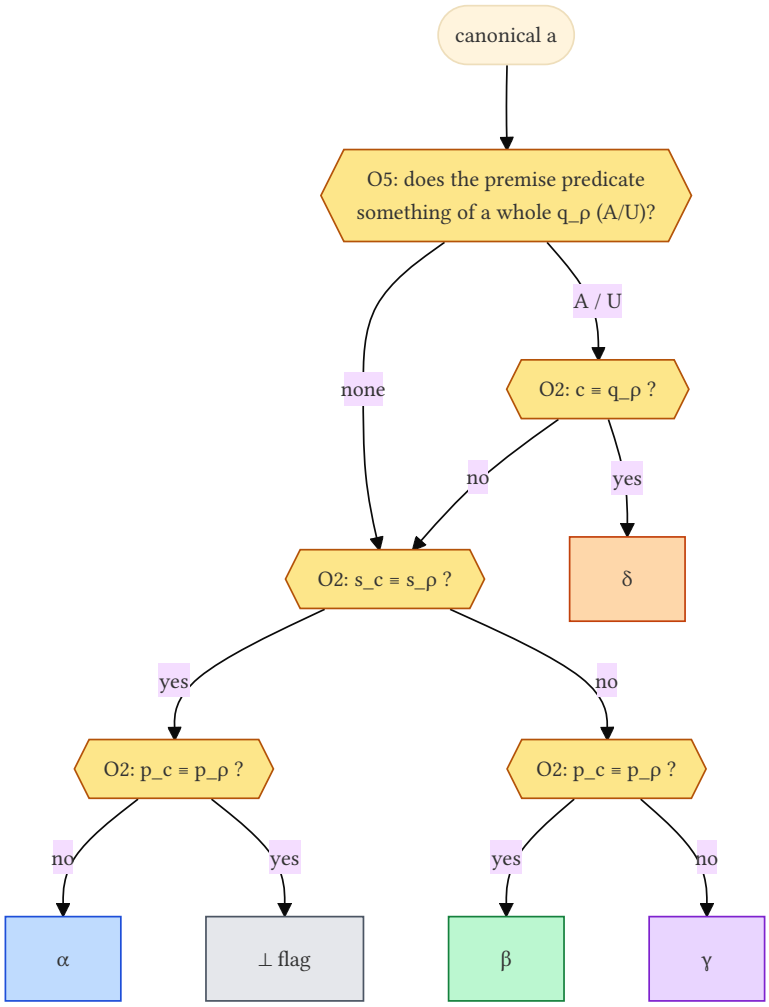
Delta covers the source-based arguments: appeals to an expert, to popularity, threats, personal attacks. Delta is structural: it is the alpha pattern, a shared subject, where the shared subject is a whole proposition and the conclusion's predicate is acceptability. Under the delta reading of step 2:

$$\delta \quad \text{iff} \quad q_\rho \neq \varnothing \wedge c \equiv q_\rho \wedge \zeta \in \mathbb{A}$$

($\varnothing$: step 1 found no such proposition.) Two judgements decide it: **O5**, whether the premise really predicates something of q_ρ as a whole, and with which ζ (values A, U, none); and **O2**, whether q_ρ is the conclusion. Delta is checked first because its reading re-splits both statements; if it holds, the ordinary split isn't used. The form function f maps an argument to its form:

$$f(a) = \begin{cases} \delta & \text{if } O_5(a) \in \mathbb{A} \wedge c \equiv q_\rho \\ \alpha & \text{if } s_c \equiv s_\rho \wedge p_c \not\equiv p_\rho \\ \beta & \text{if } p_c \equiv p_\rho \wedge s_c \not\equiv s_\rho \\ \gamma & \text{if } s_c \not\equiv s_\rho \wedge p_c \not\equiv p_\rho \\ \bot & \text{if } s_c \equiv s_\rho \wedge p_c \equiv p_\rho \end{cases}$$

⊥ (both shared) is a rephrase or begging the question; ATIP has no form for it, so we flag it and stop. As a decision tree, with oracles as **hexagons** and each form in its own colour:



The oracles return probabilities, so $\equiv$ holds with some probability rather than outright. Treating the sameness judgements as independent, each leaf's probability is the product along its path. In the sketch, `O2(t1, t2)` is $P(t_1 \equiv t_2)$:

```
def form(a):
    zeta = O5(a)              # distribution over {"A", "U", "none"}
    d  = (1 - zeta["none"]) * (O2(a.c, a.q_ρ) if a.q_ρ else 0)  # P(δ)
    ss = O2(a.s_c, a.s_ρ)     # P(subjects are the same)
    pp = O2(a.p_c, a.p_ρ)     # P(predicates are the same)
    return {
        "δ": d,
        "α": (1 - d) * ss       * (1 - pp),
        "β": (1 - d) * (1 - ss) * pp,
        "γ": (1 - d) * (1 - ss) * (1 - pp),
        "⊥": (1 - d) * ss       * pp,
    }
```

Examples, one per outcome:

argument	shared	form
the minister did a great job because *she* raised the university budget	subject (she ≡ the minister)	α

argument	shared	form
cycling on the lawn is prohibited because all activities on the lawn are prohibited	predicate (is prohibited)	β
peace can be started by the truth because war can be started by lies	nothing	γ
the economy has grown, because Eurostat said so	the proposition: q_ρ = "so" = "the economy has grown" $\equiv c$; $\zeta = A$	δ
the minister did well because the minister did a good job	both	⊥

Running example: $P(\delta) = 0.03$, $P(s \equiv) = 0.95$, $P(p \equiv) = 0.04$, so $P(\alpha) \approx 0.88$, $P(\bot) \approx 0.04$, $P(\gamma) \approx 0.05$.

1.5.5. Step 4: substance

The values the substance can take depend on the form:

- **α**: the type of each predicate. **F** (fact): a claim checkable by observation or evidence, whether or not it is true. **V** (value): a judgement against some standard (aesthetic, moral, legal, …). **P** (policy): a recommendation to act, often with "should". Oracle **O3**.
- **β**: the quantity of each subject. **s** (singular): one individual. **p** (particular): some members of a group. **u** (universal): all of them. Oracle **O4**. (This p is unrelated to the predicates p_c, p_ρ.)
- **γ**: **i** (identical) if the subjects relate to each other as the predicates do, otherwise **d** (different). Oracle **O7**.
- **δ**: the perspective of the premise's source. **I**: first person, the arguer. **II**: second person, the addressee (threats, rewards, their commitments). **III**: third person (an expert, a witness, the public). Oracle **O6**.

For α and β the substance is a pair, written conclusion first: **VF** is a value conclusion from a factual premise. The substance function S:

$$S(a) = \begin{cases} \big(O_3(p_c), O_3(p_\rho)\big) \in \{F, V, P\}^2 & \text{if } \alpha \\ \big(O_4(s_c), O_4(s_\rho)\big) \in \{s, p, u\}^2 & \text{if } \beta \\ O_7\big(R_s, R_p\big) \in \{i, d\} & \text{if } \gamma \\ O_6(Z) \in \{\mathrm{I}, \mathrm{II}, \mathrm{III}\} & \text{if } \delta \end{cases}$$

For gamma, R_s is the relation between the subjects (s_ρ to s_c) and R_p the relation between the predicates (p_ρ to p_c); the substance is i iff $R_s = R_p$. O7 judges that comparison, so its input is the pair of relations, not the whole argument. For delta, the perspective is read off the premise's predicate Z (who says, threatens, commits, …).

Rule baselines:

```
def O3(p):  # predicate type: F / V / P
    if p.modal in {"should", "ought", "must"} or p.is_imperative:
        return "P"
    if p.head in EVALUATIVE or any(m in EVALUATIVE for m in p.modifiers):
        return "V"
    return "F"

def O4(s):  # subject quantity: s / p / u
    if s.det in {"all", "every", "each"} or s.is_generic_plural:
        return "u"
    if s.det in {"some", "a few", "many", "most"}:
        return "p"
    return "s"  # proper noun, "the X", pronoun, activity

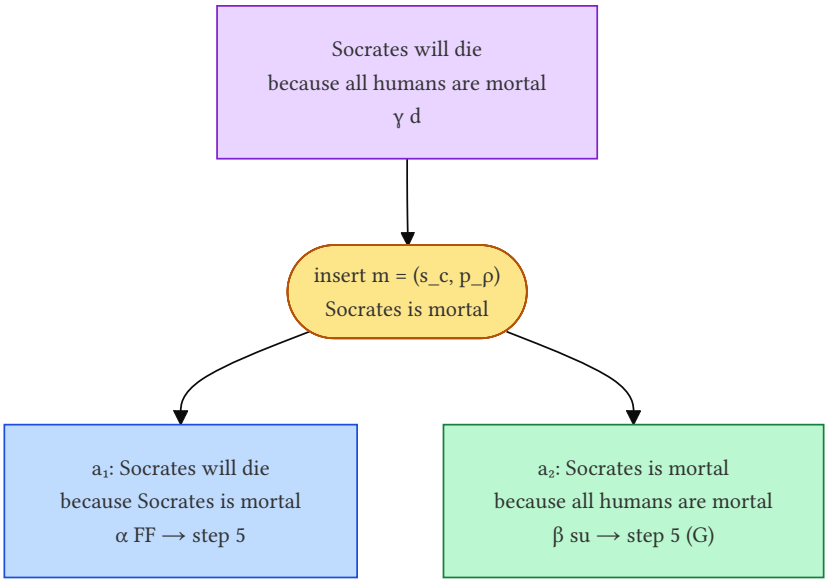
def O6(Z):  # perspective of the source in Z: I / II / III
    match Z.source.person:
        case 1: return "I"
        case 2: return "II"
        case _: return "III"
```

Examples:

form	input	oracle says	substance
α	`p_c` “did a great job” `p_ρ` “raised the university budget”	V (“great” is evaluative) F	**VF**
β	`s_c` “cycling on the lawn” `s_ρ` “all activities on the lawn”	s u (“all”)	**su**
γ	R_s: war ⟶ peace R_p: lies ⟶ truth	both are OPPOSITE, so $R_s = R_p$	**i**
δ	Z “was said by Eurostat”	third person	**III**

Condensed gamma. A gamma argument with substance d is usually an argument with a missing middle step. ATIP says to try inserting it before concluding that the premise is irrelevant, which is the type *non sequitur* (`Non`). Build the middle statement $m = (s_c, p_\rho)$ (“a is Y”), then type $a_1 = (c, m)$ as α and $a_2 = (m, \rho)$ as β. Depth one; no further recursion. The open problem is scoring whether the insertion makes the premise relevant (Section 1.9). The result is two output records, linked.

Example: “Socrates will die because all humans are mortal”. Nothing is shared (Socrates ≠ all humans, will die ≠ are mortal), so γ, and the two relations aren’t the same, so O_7 says d. Insert m = “Socrates is mortal”:



1.5.6. Step 5: lever

The lever is the argument's third statement (Section 1.1), so step 5 chooses a statement, not just a keyword. Write F for the form and t for the substance. $K(F,t)$ is the set of keywords the PTA lists for that form and substance (Section 1.12); for example $K(\alpha, \mathrm{VF}) = \{\mathrm{CRITERION}, \mathrm{DEFINITION}, \mathrm{SUBSUMED}\}$. Each keyword k names a relation, and the candidate levers are its instances ℓ_k on the argument's own terms:

$$L(F,t) = \{\ell_k : k \in K(F,t)\}$$

The terms a lever relates are fixed by the form:

form	ℓ_k	relates
α	$k(Y,X)$	premise predicate, conclusion predicate: $(\mathrm{Pr},\mathrm{Pr})$
β	$k(a,b)$ or $k(b,a)$	the subjects: $(\mathrm{Su},\mathrm{Su})$; the keyword fixes the order (a is ANALOGOUS to b; b is an EXAMPLE of a)
γ	$k(b,a) \wedge k(Y,X)$	one relation holding on both pairs
δ	$k(Z,\zeta)$	premise predicate, acceptability: $(\mathrm{Pr},\mathbb{A})$

The candidates are deductive. $K(F,t)$ is empty for α FV/VP/FP. The PTA is open to new types, so a lever outside the table is reported as `novel` rather than forced into a listed type.

1. **Cue override.** If the context states a keyword ("is a SIGN of", "is ANALOGOUS to") between the argument's own terms, return that keyword with probability 1.
2. **Otherwise oracle O8.** Write out each candidate lever statement ℓ_k as a sentence, the hypothesis h_k ("rising prices are a CAUSE of the strike"). Score each with an NLI model[7] as entailment of h_k by the context. Normalise over $K(F,t) \cup \{\mathrm{none}\}$ with a softmax;[8] `none` above a threshold gives `novel`.

$$l(a) = \mathrm{softmax}_{k \in K(F,t) \cup \{\mathrm{none}\}}\ \mathrm{entail}(\mathrm{ctx}, h_k)$$

[7] Natural language inference: given a text and a hypothesis, the model scores whether the text entails the hypothesis (makes it follow), contradicts it, or neither.

[8] softmax turns a list of scores into probabilities that sum to 1, keeping their order: $\mathrm{softmax}(z)_k = e^{z_k} / \sum_j e^{z_j}$.

```
def lever(a, form, substance):
    K = LEVERS[form][substance]              # table lookup
    if not K:
        return {"novel": 1.0}
    for k in K:
        if cue(k).links(a.terms, a.ctx):     # e.g. "is a SIGN of"
            return {k: 1.0}
    h = {k: fill(TEMPLATE[k], a) for k in K}
    h["none"] = "none of these"
    return softmax({k: entail(a.ctx, h_k) for k, h_k in h.items()})
```

Running example, α VF. The candidates and their filled-in hypotheses (Y = "raising the university budget", X = "doing a great job"):

k	h_k	P
CRITERION	raising the university budget is a criterion for doing a great job	0.62
SUBSUMED	raising the university budget can be subsumed under doing a great job	0.21
DEFINITION	raising the university budget is the definition of doing a great job	0.09
none		0.08

With a cue instead: "raising the budget is the very *definition* of a great job" skips the model and returns DEFINITION with probability 1.

1.5.7. Step 6: name

$$n : (F, t, k) \to N \cup \{\text{novel}\}$$

N is the set of the PTA's 50 type names. n is a pure lookup in Section 1.12, which reproduces ATIP's Table 7. `γ d` has no keyword (k is empty) and maps to `Non`, which is a type, not a missing value. Within a form the keyword alone fixes the name, as the PTA names types after the lever's KEYWORD; t is kept as a consistency check (a keyword outside $K(F, t)$ is an error upstream).

```
NAMES[("α", "VF", "CRITERION")]  # "argument from criterion (Cr)" -> "α VF Cr"
NAMES[("β", "su", "GENUS")]      # "argument from genus (G)"      -> "β su G"
NAMES[("γ", "d",  None)]         # "non sequitur (Non)"           -> "γ d Non"
NAMES.get(("α", "FV", None))     # None                           -> "novel"
```

1.5.8. Composition

The tree is small: at most 4 forms × 9 substances × 10 levers. Enumerate every path and multiply its oracle probabilities to get an exact distribution over codes:

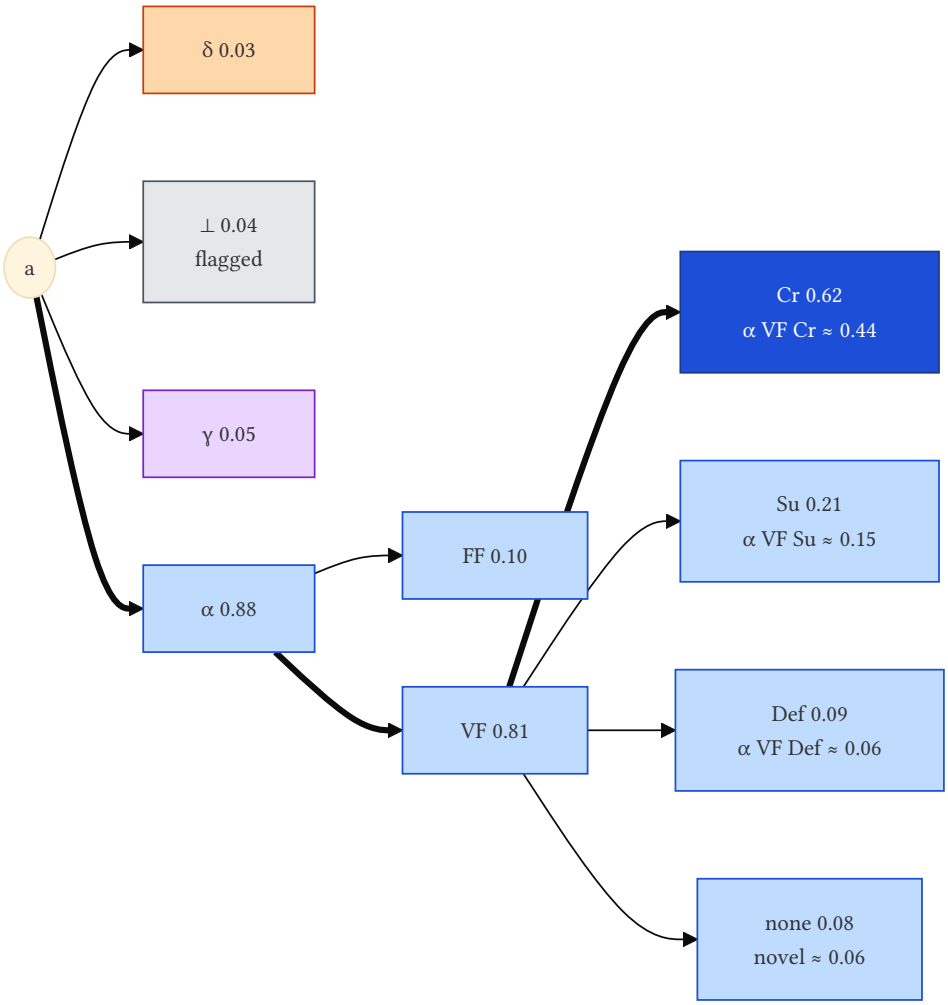
$$P(\text{code}) = \sum_{\text{paths} \to \text{code}} \prod_i P(O_i = v_i)$$

Here v_i is the answer oracle O_i gives on that path. The output record's `code` is the most likely code, `alternatives` the runners-up, and `confidence` the top probability. The product assumes the oracles are independent; the lab checks whether the resulting confidences are calibrated.[9] Paths whose probability falls below a threshold are pruned (not explored further).

[9] Confidence matches accuracy: of all the codes given confidence 0.8, about 80% are right.

```
def classify(a):
    dist = Counter()
    for F, pF in form(a).items():
        if F == "⊥":
            dist["⊥"] += pF
            continue
        for t, pt in substance(a, F).items():
            for k, pk in lever(a, F, t).items():
                p = pF * pt * pk
                if p >= PRUNE:
                    dist[code(F, t, k)] += p
    return dist
```

Running example as a tree. Probabilities multiply down each path; the winning path is bold.



Record: `code: α VF Cr`, `confidence: 0.44`, `alternatives: [α VF Su, α VF Def]`. A confidence that low says the label should be reviewed before it is trusted.

The lever and naming tables are data, not code: one file read by both the lab (Python) and the game (Rust).

1.6. Oracles

Each oracle has a rule baseline and at least one model arm. Several of the models are already in use in the reading-notes lab, a sibling project that maps passages of text to concepts.

id	question	values	rule baseline	model arm
O1	where are c, ρ, the subjects, the predicates and the markers?	spans	dependency parse + marker lexicon	GLiNER2[10] extraction
O2	$t_1 \equiv t_2$?	yes/no	lemma[11] match after coreference[12]	embedding cosine[13], cross-encoder[14]
O3	predicate type	F/V/P	modals and imperatives $\longrightarrow$ P; evaluative lexicon $\longrightarrow$ V; else F	GLiNER2 classification / small classifier
O4	subject quantity	s/p/u	determiners and quantifiers from the parse; proper noun $\longrightarrow$ s	rarely needed
O5	does the premise predicate something of a whole proposition q_ρ, accepting or rejecting it?	A/U/none	reporting verbs with a clausal or anaphoric object, "according to", conditionals on q	GLiNER2 classification
O6	whose perspective is Z?	I/II/III	grammatical person of the source	rarely needed
O7	is $R_s = R_p$?	i/d	—	NLI
O8	which lever?	$K(F,t) \cup \{\text{none}\}$	cue words	NLI over filled-in levers
O9	which rewordings could make a term shared?	candidate pairs (c', ρ') (text)	passive $\leftrightarrow$ active from the parse	generative model
O10	does a rewording say the same as the original?	probability	—	NLI, both directions

Reuse: the reading-notes lab already runs embedding models (usable for O2) and GLiNER2 (O1, O3, O5). Two things are new:

- **An NLI arm** (O7, O8): small DeBERTa-v3 NLI models (xsmall to base) run on CPU and export to ONNX,[15] so they fit that lab's latency harness.
- **A dependency parser** for the rule baselines (spaCy `en_core_web_sm` in the lab). Most of the deductive work runs on it.

[10] A small encoder that extracts spans and classifies text against labels given at run time.
[11] A word's dictionary form: "argued" $\longrightarrow$ "argue".
[12] Working out which expressions refer to the same thing: "Kant … he".
[13] Turn each term into a vector and score the angle between them; near 1 means similar meaning.
[14] A model that reads both terms together and scores them as a pair; slower than embeddings, usually more accurate.
[15] A portable model format, run by ONNX Runtime in the lab and `ort` in Rust.

1.7. Gold labels and evaluation

The gold labels are features on each `Connector` (Section 1.4.1). The classifier's output record has the same fields, so every gold value can be compared with what the pipeline produced:

feature	values
`conclusion subject` · `conclusion predicate` · `premise subject` · `premise predicate`	the canonical terms as text: a, X, b, Y in the lever templates (the premise repeats a for α and X for β); for δ, q, `[is acceptable]` or `[is unacceptable]`, q, Z
`transformations`	the wording changes, each `kind (statement)` from a closed tagset: anaphora, explication, illocution substitution, passivization, activization, nominalisation, same term; once each, unordered
`form`	`alpha` · `beta` · `gamma` · `delta`
`substance`	the substance codes, prefixed by their form (`α VF`, `β su`, `γ i`, `δ III`, …)
`lever`	the lever keyword (`CAUSE`, `GENUS`, …), `none` for γ d, or `novel` with the wording in `note`
`concepts`	what the argument is about: one or more tags, each a broad `concept` or a writer's own `concept/sense` (`being`, `being/scotus`, `being/heidegger-ontological`)

Concept tags are references, not a theory of concepts: a tag names what an argument is about at the level of a glossary entry, and a sense records a writer's own version where the passage depends on it. Whether two writers share a concept stays a labelling decision, made per passage. Tags are lowercase, because INCEpTION may compare tag names case-insensitively. The tagset is open, so tags are added while labelling; together they become the concept list the models later choose from. Constraints can't relate a concept to its senses in a multi-valued feature, so the shared prefix does that job: typing `being` lists every `being/…` sense.

Substance tags carry their form because INCEpTION may compare tag names case-insensitively, which would merge `pp` with `PP` and `i` with `I`. INCEpTION *constraints* show the substances valid for the chosen form, and the levers valid for the chosen substance, first and in bold; they don't forbid the others, so the converter rejects invalid combinations. The layers and constraints are generated from the type table (Section 1.12) by `scripts/pta_schema.py` in the lab.

The name is derived from the three, so it isn't stored. The features map one-to-one onto pipeline stages (the canonical terms and transformations onto steps 1–2, the rest onto steps 3–5), which allows **stage-wise evaluation**: run the pipeline with gold values substituted for everything upstream of a stage, and that stage's errors are its own. Canonical terms are compared by meaning, not string: span overlap for terms taken from the text, entailment both ways (as O10 checks rewrites) for reworded ones. Transformations are compared as sets. End to end, measure code top-1/top-3[16] and calibration. Statistics: each arm is compared pairwise against one baseline, with confidence intervals from resampling by passage.

[16] Whether the gold code is the most likely code / among the three most likely.

A second human pass on a sample gives the ceiling: ATIP is hard for people too, and no arm should be expected to beat human agreement.

Pilot corpus. The first corpus is one short argumentative excerpt from each of about 30 writers across traditions (ancient, Buddhist, patristic and scholastic, early modern, German idealist, phenomenological, analytic, critical, psychoanalytic, Kyoto School, and speeches), plus the ten single-argument examples from the PTA website, which test typing on their own. One writer's style shouldn't shape the whole corpus. The pilot finds where the pipeline breaks; it is too small to rank arms.

- **Split by writer.** Each writer in turn is the test set and the rest form the dev set, so every annotation is tested once and no example helps to classify its own writer.
- **What the dev set is for.** O9′s generative arm takes 3–8 dev cases as in-context examples (original pair, the labeller's rewording, its transformations), chosen by similarity to the current argument. The other oracles are used zero-shot; the dev set sets their thresholds (O2, O10) and calibrates their confidences (O7, O8, GLiNER2). Rule baselines may be fixed against the dev set only.
- **Rewordings as gold.** The canonical terms and transformations hold the labeller's rewording; `note` holds only remarks. O9′s examples and its evaluation read the rewording; the subject/predicate splits are gold for O1 and O2.
- **First run.** The classifier runs on the gold connectors, stage-wise and end to end. Finding the connectors automatically (the document-analysis procedure) has no oracles yet and comes later.

Records quote the lab's source texts, which are excerpts of published books, so they are never committed.

1.8. In-game use (sketch)

The player argues with a non-player character (NPC); this pipeline types the player's arguments. The result is one input to **key extraction**: the game's step that decides which of the game's keys an argument unlocks.

What changes from the lab:

- **No gold spans.** O1 must find the claim, the reason and which is which, from the player's line *and* the dialogue: the conclusion is often the NPC's last claim, which the player supports or attacks.
- **Short, informal text.** The rule baselines will do worse than on books; the lab should keep a set of player-style notes to measure this separately.
- **Runtime.** Rust with `ort`, on a throttled CPU. Every model arm must export to ONNX, and the rule baselines must be small enough to port. The tables are already shared data (Section 1.5.8).
- **Rewriting in-game.** O9′s LLM arm is too slow and too large (400 MB at 0.5B, 4-bit) for a reply that should feel immediate. In-game O9 uses the rule and seq2seq arms, gated as in Section 1.5.3, and may run after the player's line rather than before the reply. NPC arguments are analysed offline. The LLM arm stays in the lab, where its reviewed outputs can train the seq2seq arm.

What key extraction gets, per argument:

- the code distribution;
- the **bound terms**: which spans filled a, b, X, Y, q, each run through the reading-notes lab's interpreters, which map a span to a concept id.

With the bound terms, an argument becomes a structured claim about concepts: "the player argued that concept Y is a CAUSE of concept X". That is the natural thing for a key to gate on, e.g. a key that needs concept X introduced *and* argued for by a lever of some type. The quality step slots in between: it reads the same record and can veto. How keys combine these is left open.

1.9. Risks and open questions

- **Subject/predicate splits in philosophical prose.** Nominalisations and long clauses will break the parse baseline. Expected to be the largest source of O1 error.
- **Sameness is the pivot.** O2 decides form, and form decides every later candidate set. Errors there cascade; stage-wise evaluation will show how much.
- **Rewriting decides form.** Allowed rewordings are loose enough that one argument can come out α, β or γ (Section 1.5.3). O9 and O10 make the choice explicit and reviewable, but their errors move arguments between forms; the lab should measure how often O9′s candidates include the labeller's rewording.
- **Condensed gamma.** No good scorer for "the insertion makes the premise relevant" yet. First version: try it, report both readings, lower the confidence.
- **Explication in free text.** Players will leave the link implicit. The lever is always reconstructed anyway (O8), but a missing *premise* isn't handled. Deferred until there is player-style data.
- **Independence assumption** in the composition. Checked by calibration; if it fails, recalibrate the composed score rather than modelling dependencies.
- **A parser in Rust.** The rule baselines assume a dependency parser. The game needs one that runs under `ort` or in pure Rust; not yet surveyed.
- **Training data.** No PTA 3.1 corpus exists that I know of; Visser et al. 2021 uses the old 36-type labels and would need remapping. Fine-tuning stays deferred, as in the reading-notes lab.

1.10. Suggested order

1. Tables as data; form, substance, lever and naming as pure functions over given oracle values; composition. Testable with hand-made values.
2. Gold features in INCEpTION; stage-wise evaluation with gold oracles.
3. Rule baselines (parser, lexicons).
4. Model arms: embeddings (O2), GLiNER2 (O1, O3, O5), NLI (O7, O8).
5. Player-style data and in-game O1.

1.11. References

- Wagemans, J.H.M. (2026). *Argument Type Identification Procedure (ATIP) – Version 6.* https://www.periodic-table-of-arguments.org/argument-type-identification-procedure
- Wagemans, J.H.M. *Periodic Table of Arguments, version 3.1*, with worked examples of every type. https://www.periodic-table-of-arguments.org/pta-3-1-clickable
- Hinton, M., Wagemans, J.H.M. (2023). "How persuasive is AI-generated argumentation? An analysis of the quality of an argumentative text produced by the GPT-3 AI text generator." *Argument & Computation* 14(1), 59–74. https://doi.org/10.3233/AAC-210026 Local copy in `.docs/reference/`.
- Hinton, M., Wagemans, J.H.M. (2022). "Evaluating reasoning in natural arguments: A procedural approach." *Argumentation* 36, 61–84. The type-specific lever questions.
- Visser, J., Lawrence, J., Reed, C., Wagemans, J., Walton, D. (2021). "Annotating Argument Schemes." *Argumentation* 35, 101–139.
- Chesñevar, C., McGinnis, J., Modgil, S., Rahwan, I., Reed, C., Simari, G., South, M., Vreeswijk, G., Willmott, S. (2006). "Towards an Argument Interchange Format." *Knowledge Engineering Review* 21(4), 293–316.

1.12. Appendix: the 50 PTA 3.1 types

From ATIP's Table 7. The lever column is the lever statement's pattern, with the keyword in capitals. For β, the substance is any one of those listed.

form	substance	lever	name	symbol
α	FF	Y is a SIGN of X	argument from sign	Sig
α	FF	Y is a CAUSE of X	argument from cause	Cau
α	FF	Y is an EFFECT of X	argument from effect	Ef
α	FF	Y is CORRELATED with X	argument from correlation	Cor
α	FF	Y is a MOTIVE for X	argument from motive	Mot
α	FF	Y EXCLUDES X	argument from incompatibility	Inc
α	VF	Y is a CRITERION for X	argument from criterion	Cr
α	VF	Y is the DEFINITION of X	argument from definition	Def
α	VF	Y can be SUBSUMED under X	argument from subsumption	Su
α	VV	Y is a NORM for X	argument from norm	Nor
α	VV	Y is an INSTANTIATION of X	argument from instantiation	Ins
α	VV	Y is a DISQUALIFIER of X	argument from disqualifier	Dis
α	PF	Y is a PRAGMATIC MOTIVATION for X	pragmatic argument	Pra
α	PF	Y is a PRIOR INVESTMENT in X	argument from waste	W
α	PF	Y is the FEASIBILITY of X	argument from possibility	Pos
α	PV	Y is a PRINCIPLED MOTIVATION for X	argument from principle	Pri
α	PV	Y is an EVALUATIVE MOTIVATION for X	argument from evaluation	Ev
α	PP	Y is a DEONTIC MOTIVATION for X	deontic argument	Deo
α	PP	Y is a TELEOLOGIC MOTIVATION for X	teleologic argument	Tel
β	ss, pp, uu	a is ANALOGOUS to b	argument from analogy	A
β	ss, pp, uu	a is SIMILAR to b	argument from similarity	Sim
β	ss, pp, uu	a is COMPARABLE to b	argument from comparison	Cpa
β	ss, pp, uu	a is EQUAL to b	argument from equality	Eq
β	ss, pp, uu	a is described METAPHORICALLY as b	metaphoric argument	Me
β	ss, pp, uu	a is COMPOSED of b	argument from composition	Cpo
β	ss, pp, uu	a is a PART of b	argument from division	Div
β	ss, pp, uu	a RANKS BELOW b	*argumentum a maiore*	Ma
β	ss, pp, uu	a RANKS ABOVE b	*argumentum a minore*	Mi
β	ss, pp, uu	a OWES RECIPROCITY to b	argument from reciprocity	R
β	ps, us, up	b is an EXAMPLE of a	argument from example	Exa
β	ps, us, up	b is a SPECIES of a	argument from species	Sp
β	sp, su, pu	a belongs to the TOTALITY of b	argument from totality	To
β	sp, su, pu	a belongs to the GENUS of b	argument from genus	G

form	substance	lever	name	symbol
γ	i	b is SYNONYMOUS to a and Y is SYNONYMOUS to X	argument from synonyms	Sy
γ	i	b is the OPPOSITE of a and Y is the OPPOSITE of X	argument from opposites	Op
γ	i	b is DERIVATIVE of a and Y is DERIVATIVE of X	argument from derivatives	Der
γ	i	b is INTEGRATIVE of a and Y is INTEGRATIVE of X	argument from integratives	Int
γ	i	b is the CONVERSE of a and Y is the CONVERSE of X	argument from converses	Cve
γ	d	—	non sequitur	Non
δ	I	Z is ETHOTICALLY AUTHORITATIVE for A	ethotic argument	Eth
δ	II	Z is CHARACTERALLY AUTHORITATIVE for U	*argumentum ad hominem*	H
δ	II	Z is a NEGATIVE MOTIVATION for A	*argumentum ad baculum*	B
δ	II	Z is an OFFERED REWARD for A	*argumentum ad carotam*	Car
δ	II	Z COMMITS the addressee to A	argument from concession	Cce
δ	II	Z is an UNDESIRABLE IMPLICATION supporting U	*argumentum ad consequentiam*	Cse
δ	III	Z is EPISTEMICALLY AUTHORITATIVE for A	argument from expert opinion	Exp
δ	III	Z is MORALLY AUTHORITATIVE for A	argument from moral authority	Mor
δ	III	Z is TESTIMONIALLY AUTHORITATIVE for A	argument from testimony	Tes
δ	III	Z is POPULARLY AUTHORITATIVE for A	*argumentum ad populum*	Pop
δ	III	Z is a PRESUMPTION in favor of A	*argumentum ad ignorantiam*	Ig